

CONNECTA

Sistema Inteligente de Gestión Veterinaria · B2B

Python 3.11 · Flask 3.1.3 · MongoDB 7 · WhatsApp Business

Bitácora Técnica

Avance 9 — Guía Práctica N°9

Programación Orientada a Objetos (POO) Nivel I

Curso:	Diseño Funcional — FESC
Estudiante:	Erick Santiago Ocampo Marciales
Fecha:	15 de mayo de 2026
Commit:	4401d08 → branch: main
Archivos:	13 modificados · +801 líneas / -368
Stack:	Python · Flask · MongoDB · Socket.IO

github.com/AntionART/CONNECTA

1. Resumen Ejecutivo

El Avance 9 transforma los modelos de CONNECTA de "namespaces con métodos estáticos" a clases OOP completas con estado, comportamiento y encapsulamiento. El patrón Template Method aplicado en BaseModel permite que `find_by_id()` retorne instancias tipadas (Pet, Appointment, Conversation, Message) sin cambiar las rutas de Flask que ya las usaban.

Métricas del cambio

Indicador	Valor
Entidades refactorizadas	4 (Pet, Appointment, Conversation, Message)
Archivos modificados	13
Líneas añadidas	+801
Líneas eliminadas	-368
@property / setters	9 (con validación)
Métodos migrados a instancia	2 (enriquecer_dashboard, enriquecer_api)
Atributos privados (_xxx)	7 en total (PII + estado)
Commit hash	4401d08 — rama main

2. Actividades Implementadas

ACTIVIDAD 1 — Modelado de Entidades (`__init__`, `from_db`, `to_dict`, `__str__`, `__repr__`)

Cada entidad ahora tiene un constructor explícito que declara todos sus atributos de instancia. `from_db()` actúa como factory desde MongoDB; `to_dict()` serializa de vuelta a JSON. Los atributos PII y de estado crítico se prefijan con `_` para encapsulamiento según convención Python (PEP 8).

Clase	Archivo	<code>__init__</code>	<code>from_db</code>	<code>to_dict</code>	<code>__str__</code>	Privados
Pet	models/pet.py	L23	L49	L68	L93	<code>_owner_phone</code> , <code>_owner_name</code>
Appointment	models/appointment.py	L30	L51	L68	L95	<code>_status</code>
Conversation	models/conversation.py	L23	L54	L73	L106	<code>_phone_number</code> , <code>_status</code> , <code>_unread_count</code>
Message	models/message.py	L22	L44	L63	L82	<code>_status</code>
BaseModel	models/base.py	—	L22	—	—	default <code>from_db</code> ; <code>find_by_id</code> → <code>cls.from_db</code>

ACTIVIDAD 2 — Métodos y Properties

@property con setter implementados para atributos sensibles. Dos funciones sueltas de rutas migradas como métodos de instancia con `self`.

Property / Método	Clase	Línea	Descripción
<code>owner_phone</code> + setter	Pet	L108	Valida str no vacío, <code>strip()</code>
<code>owner_name</code> + setter	Pet	L121	Normaliza: <code>strip()</code> o "
<code>pertenece_a(phone)</code>	Pet	L132	Método instancia — compara PII
<code>status</code> + setter	Appointment	L115	Valida contra <code>STATUSES[]</code>
<code>enriquecer_dashboard()</code>	Appointment	L133	MIGRADO desde <code>dashboard.py</code>
<code>enriquecer_api()</code>	Appointment	L148	MIGRADO desde <code>appointments.py</code>
<code>phone_number</code> (read-only)	Conversation	L124	PII inmutable, sin setter
<code>status</code> + setter	Conversation	L130	Valida 'open'/'closed'
<code>unread_count</code> (read-only)	Conversation	L144	Solo BD vía <code>update_last_message</code>

status (read-only)	Message	L98	Pipeline entrega WhatsApp
--------------------	---------	-----	---------------------------

ACTIVIDAD 3 — Reingeniería del Sistema

Las rutas Flask y el motor lógico (webhook, dashboard) usan ahora instancias tipadas: `obj.to_dict()` reemplaza `serialize_doc(dict_crudo)`. El webhook accede a `conv.id` y `conv.contact_name` como atributos; el dashboard llama `apt.enriquecer_dashboard()` en lugar de una función suelta.

Archivo	Cambio clave	Línea
models/base.py	<code>from_db()</code> Template Method; <code>find_by_id()</code> delega	L22, L27
routes/webhook.py	<code>conv.id</code> , <code>conv.contact_name</code> , <code>ObjectId(msg.id)</code>	L83, L87, L122
routes/dashboard.py	<code>Appointment.from_db(doc).enriquecer_dashboard()</code>	L80
routes/api/pets.py	<code>[p.to_dict() for p in pets]</code>	L13
routes/api/appointments.py	<code>apt.enriquecer_api()</code> — método del objeto	L29
routes/api/conversations.py	<code>[c.to_dict() for c in conversations]</code>	L24
routes/api/messages.py	<code>conv.phone_number</code> ; <code>msg.to_dict()</code>	L58, L69
utils/helpers.py	<code>msg.to_dict()</code> , <code>updated_conv.to_dict()</code> en Socket.IO	L111, L113

3. Diagramas Mermaid — Nueva Arquitectura OOP

Diagrama 1 — Jerarquía de Clases (Mermaid classDiagram)

Muestra la herencia BaseModel → Pet, Appointment, Conversation. Signos: + público, - privado (_atributo). Se omite Message de la jerarquía visual para simplificar; Message tiene la misma estructura OOP.

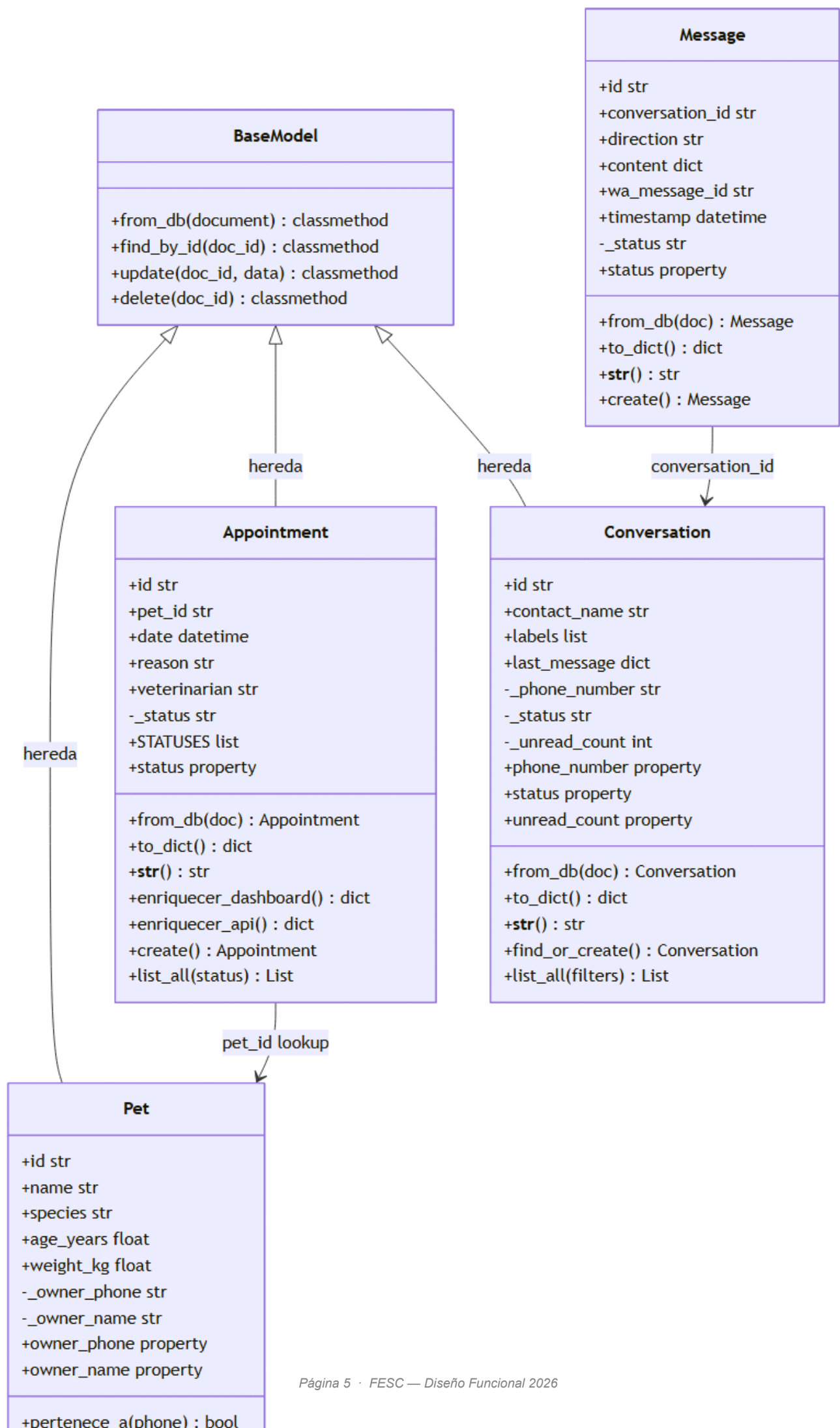


Diagrama 2 — Patrón Template Method en BaseModel (flowchart)

`BaseModel.find_by_id()` llama a `cls.from_db(doc)` vía MRO de Python. Si el `doc` es `None` retorna `None` (→ 404 en la ruta). Si existe, la subclase construye su instancia tipada; la ruta llama `obj.to_dict()` para serializar a JSON.

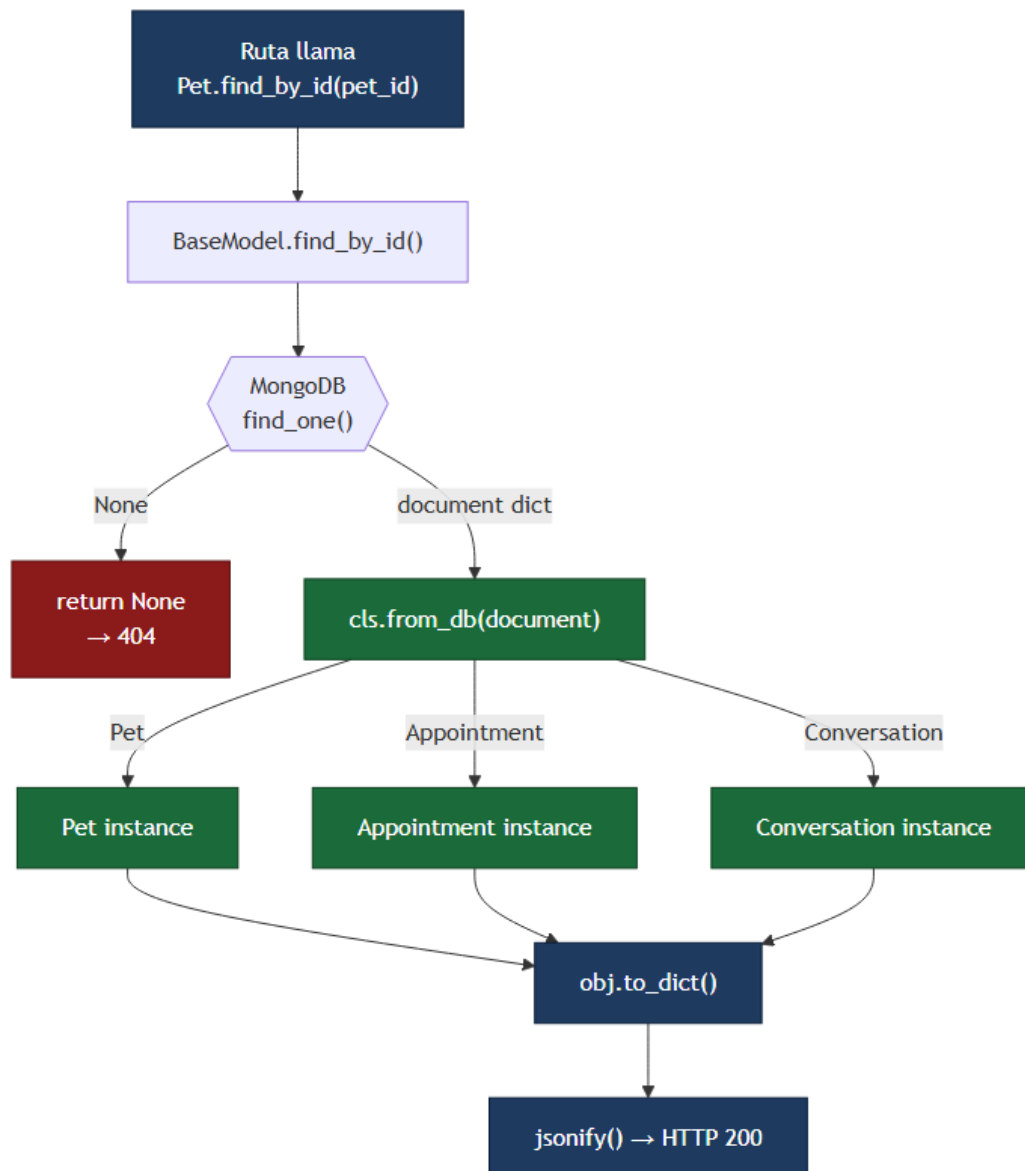


Fig. 2 — Template Method: `find_by_id()` → `cls.from_db()` → `obj.to_dict()`

Diagrama 3 — Secuencia Webhook con Instancias OOP (sequenceDiagram)

Flujo completo de un mensaje entrante de WhatsApp: `find_or_create()` retorna una instancia `Conversation`; el webhook accede a `conv.id` y `conv.contact_name` como atributos de objeto. `Message.create()` devuelve una instancia; `helpers.py` emite `msg.to_dict()` y `conv.to_dict()` a `Socket.IO`.

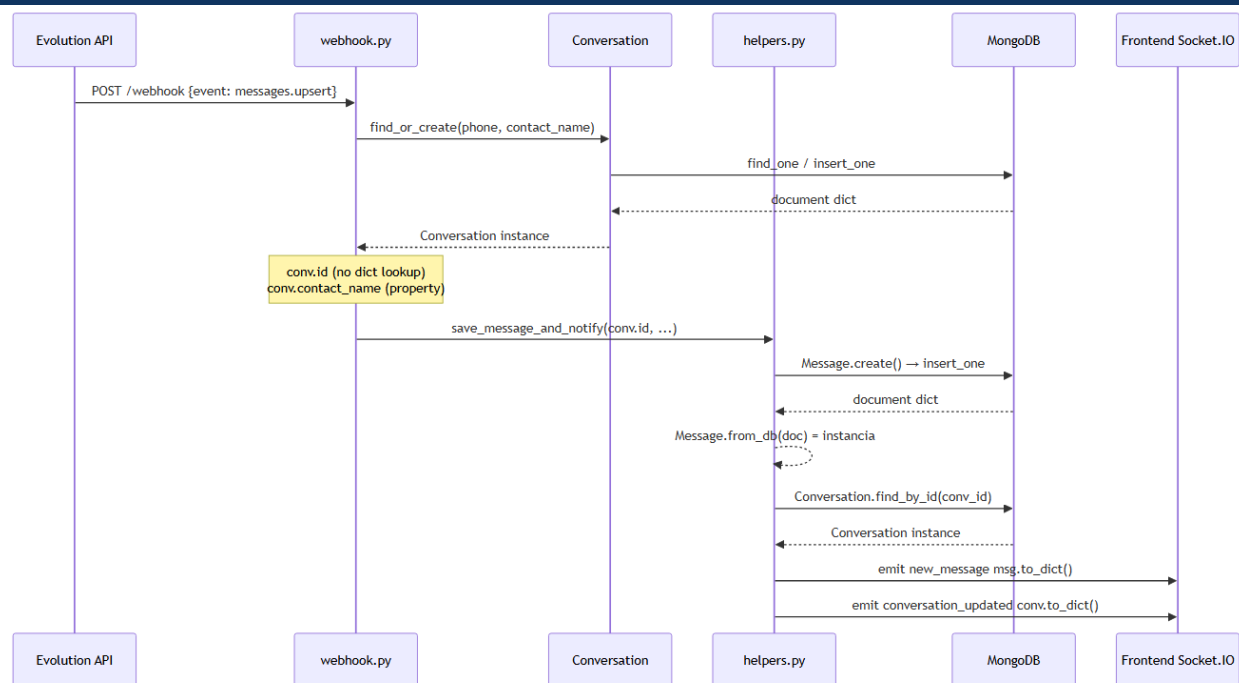
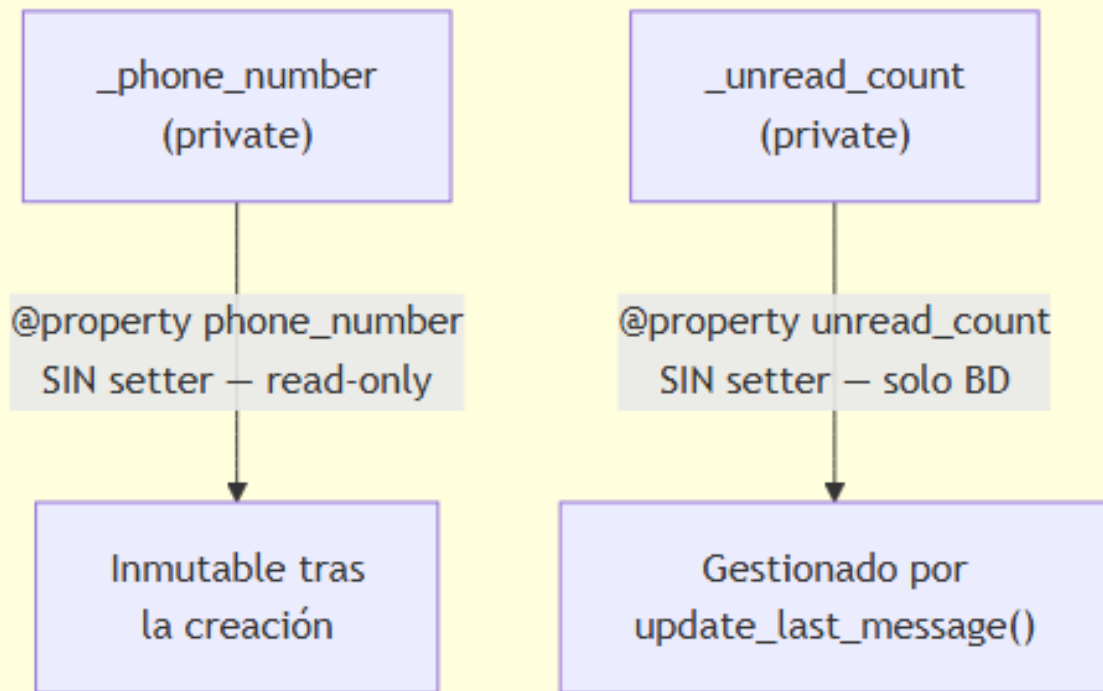


Fig. 3 — Secuencia: Evolution API → Webhook OOP → Frontend

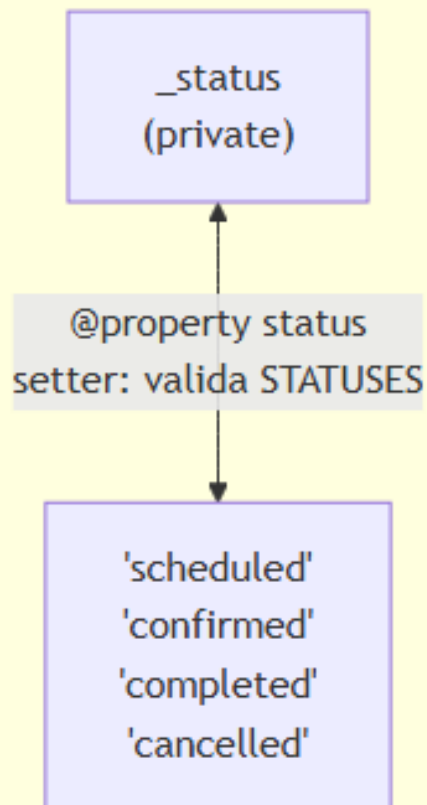
Diagrama 4 — Encapsulamiento con @property (flowchart)

Muestra cómo los atributos privados son controlados: owner_phone valida str no vacío; status valida contra STATUSES[]; phone_number en Conversation no tiene setter (PII immutable); unread_count solo es modificable por la BD.

Conversation – PII Immutable



Appointment – Estado



4. Evidencia de Código

Actividad 1 — Constructores y from_db (app/models/pet.py):

```
# [GUÍA 9 - ACTIVIDAD 1] Constructor explícito con atributos tipados
def __init__(self, name: str, species: str, breed: str,
              age_years: float, weight_kg: float,
              owner_phone: str, owner_name: str = '',
              id: str = None, created_at=None, updated_at=None):
    self.id          = id
    self.name        = name
    self.species     = species
    self.age_years   = age_years      # float - 0.5 = 6 meses
    self.weight_kg   = weight_kg      # float - decimales
    self._owner_phone = owner_phone   # PII - prefijo _
    self._owner_name = owner_name     # PII - prefijo _

# [GUÍA 9 - ACTIVIDAD 1] from_db - puente MongoDB → objeto Python
@classmethod
def from_db(cls, document: dict) -> 'Pet':
    if document is None:
        return None
    return cls(
        id=str(document['_id']),
        name=document.get('name', ''),
        owner_phone=document.get('owner_phone', ''),
        ...
    )

# [GUÍA 9 - ACTIVIDAD 1] to_dict - puente objeto Python → JSON REST
def to_dict(self) -> dict:
    return {
        '_id': self.id,
        'name': self.name,
        'owner_phone': self._owner_phone, # accede al atributo privado
        'created_at': self.created_at.isoformat() if isinstance(...) else ...,
    }
```

Actividad 2 — @property, setter y método migrado (app/models/appointment.py):

```
# [GUÍA 9 - ACTIVIDAD 2] @property con validación de ciclo de vida
@property
def status(self) -> str:
    return self._status

@status.setter
def status(self, value: str):
    if value not in Appointment.STATUSES: # ['scheduled','confirmed','completed','cancelled']
        raise ValueError(f"status inválido '{value}'. Debe ser uno de: {Appointment.STATUSES}")
    self._status = value

# [GUÍA 9 - ACTIVIDAD 2] Método migrado desde dashboard.py._enriquecer_cita_dashboard()
def enriquecer_dashboard(self) -> dict:
    from app.models.pet import Pet          # import tardío - evita circularidad
    pet = Pet.find_by_id(self.pet_id)       # find_by_id → instancia Pet (no dict)
    result = self.to_dict()
    result['pet_name'] = pet.name if pet else 'Desconocida'
```

```

return result

# [GUÍA 9 - ACTIVIDAD 2] Método migrado desde appointments.py._enrich_appointment()
def enriquecer_api(self) -> dict:
    from app.models.pet import Pet
    pet = Pet.find_by_id(self.pet_id)
    result = self.to_dict()
    result['pet'] = pet.to_dict() if pet else None
    return result

```

Actividad 3 — Reingeniería en webhook.py y dashboard.py:

```

# app/routes/webhook.py [GUÍA 9 - ACTIVIDAD 3]
# ANTES (dict crudo):
#   conv = Conversation.find_or_create(phone, name) # → dict
#   conv_id = str(conv['_id'])
#   if contact_name and not conv.get('contact_name'):
#
# DESPUÉS (instancia Conversation):
conv = Conversation.find_or_create(phone, contact_name) # → Conversation
conv_id = conv.id # atributo de instancia
if contact_name and not conv.contact_name: # @property, no dict lookup
    Conversation.update(conv_id, {'contact_name': contact_name})

# app/routes/dashboard.py [GUÍA 9 - ACTIVIDAD 3]
# ANTES: map(_enriquecer_cita_dashboard, upcoming_appointments) ← función suelta
# DESPUÉS: método del objeto
upcoming_appointments = list(map(
    lambda doc: Appointment.from_db(doc).enriquecer_dashboard(),
    upcoming_apt_docs
))

# app/routes/api/pets.py [GUÍA 9 - ACTIVIDAD 3]
# ANTES: return jsonify(serialize_docs(pets)) ← lista de dicts crudos
# DESPUÉS:
pets = Pet.list_all() # List[Pet] - instancias tipadas
return jsonify([p.to_dict() for p in pets])

```

5. Notas Arquitectónicas y Decisiones de Diseño

1. Patrón Template Method en BaseModel

`BaseModel.find_by_id()` llama a `cls.from_db(doc)`. Gracias al MRO de Python, `Pet.find_by_id()` usa `Pet.from_db()` automáticamente. Modelos auxiliares (`Label`, `Settings`) usan el `from_db()` por defecto de `BaseModel` (retorna el dict crudo), garantizando compatibilidad backward sin cambios adicionales.

2. `__init__` no hace queries a MongoDB

El constructor solo inicializa atributos en memoria. La consulta a MongoDB ocurre exclusivamente en los métodos estáticos (`create`, `find_by_id`, `list_all`). Esto mantiene separación de capas: `__init__` es Python puro, los métodos estáticos son la capa de acceso a datos.

3. `phone_number` es read-only en Conversation

El número de teléfono es la clave primaria natural de un hilo WhatsApp. Cambiar el teléfono rompería la identidad de la conversación. Se define únicamente con getter (sin setter) para imponer esta invariante de dominio.

4. Imports tardíos para evitar circularidad

`Appointment.enriquecer_dashboard()` hace ``from app.models.pet import Pet`` dentro del método. Esto evita el error de importación circular entre `appointment.py` y `pet.py`, los cuales están en el mismo paquete `models/`.

5. Funciones transversales NO migradas a clases

`validar_campos_requeridos()` (`helpers.py`) opera sobre cualquier dict. `_coincide`, `detectar_intencion()`, `generar_respuesta()` (`nlp.py`) procesan texto puro sin relación con entidades del dominio. Migrarlas hubiera violado SRP. Cada una incluye un comentario explicando por qué permanece fuera de una clase.

6. `to_dict()` como reemplazo de `serialize_doc()`

`serialize_doc()` sigue disponible para `Label` (que retorna dict crudo). Para las entidades refactorizadas (`Pet`, `Appointment`, `Conversation`, `Message`), `to_dict()` produce el mismo formato JSON — pero con manejo explícito de `datetime.isoformat()` y `str(id)`, sin depender de inspección de tipos externa.

6. Registro de Commit Git

```
$ git log -1 --format="%H %s"
4401d08 feat(avance9): implement OOP Level I - classes, properties, instance methods

Branch: main
Remote: https://github.com/AntionART/CONNECTA.git
Status: pushed [OK]

Archivos modificados (13):
ACTIVITY_MAPPING.md      | +150 líneas (sección Guía 9 + checklist)
app/models/base.py       |  from_db() + find_by_id() Template Method
app/models/pet.py        |  OOP completo + @property owner_phone/owner_name
app/models/appointment.py |  OOP completo + enriquecer_dashboard/api()
app/models/conversation.py |  OOP completo + @property (PII, estado, contador)
app/models/message.py    |  OOP completo + @property status
app/routes/webhook.py    |  conv.id, conv.contact_name, import Message
app/routes/dashboard.py  |  from_db().enriquecer_dashboard() vía map()
app/routes/api/pets.py    |  [p.to_dict() for p in pets]
app/routes/api/appointments.py | apt.enriquecer_api()
app/routes/api/conversations.py | [c.to_dict() for c in conversations]
app/routes/api/messages.py | conv.phone_number, msg.to_dict()
```

```
app/utils/helpers.py      |  msg.to_dict(), updated_conv.to_dict()
```

```
13 files changed, 801 insertions(+), 368 deletions(-)
```